RadeonAgents

Project Specification Document

Track	Track 2: Agentic AI
Project Name	RadeonAgents
Author	Shreehitha Arushan Cheekoti
Email	csarushan1729@gmail.com
GitHub	github.com/csarushan1729
Event	AMD AI DevMaster Hackathon 2026
Focus	Local Multi-Agent System optimized for AMD Radeon GPUs + ROCm
Date	August 6, 2026

1. Application Scenarios

RadeonAgents is a fully local, privacy-preserving multi-agent system designed to run on AMD Radeon GPUs with the ROCm software stack. It targets practical everyday and professional use cases where sensitive data must never leave the user's machine.

Primary Application Scenarios

- **Personal Productivity Assistant**: Creates realistic daily schedules, prioritizes tasks, and decomposes complex goals into actionable steps while respecting meetings, deep-work blocks, and personal activities.
- **Developer Copilot**: Assists software engineers with code generation, debugging, complexity analysis, and verification by leveraging tools such as a restricted Python interpreter and local filesystem access.
- **Local Knowledge Assistant (RAG)**: Answers questions over private documents (Markdown, source code, notes, internal documentation) without uploading any data to external servers.
- **Multi-Agent Workflow Automation**: Automatically routes tasks to specialized agents (Planner, Researcher, Coder, Executor) so that complex multi-step goals can be solved collaboratively and transparently.

All scenarios execute entirely on the user's AMD Radeon GPU, guaranteeing data privacy, predictable latency after model loading, and complete independence from external cloud LLM APIs.

2. Agent Architecture

RadeonAgents follows a clean layered architecture that cleanly separates user interaction, orchestration, reasoning, tools, memory, and inference. This separation enables independent optimization of each layer for ROCm.

High-Level Architecture

```
User Interface (Gradio Chat + CLI demos)
    ↓
Multi-Agent Orchestrator
(routes goals → Planner / Researcher / Coder / Executor)
```

```
    ↓
Core Agent Layer
• Hierarchical Planner    • ReAct Reasoning Loop
• Tool Calling    • Short-term + Long-term Memory
    ↓
Supporting Subsystems
• Tool Registry (calculator, Python, FS, RAG…)
• Local RAG (Chroma + sentence-transformers)
• Memory Manager (conversation buffer + vector store)
    ↓
LLM Engine (ROCm-optimized)
• Hugging Face Transformers    • 4-bit / 8-bit quantization
• SDPA attention    • Automatic AMD GPU detection via ROCm
```

Key design decisions: Hierarchical planning is performed first; each sub-task is then executed with a classic ReAct loop. Tools and memory are shared across agents for consistency. The inference engine is deliberately isolated so that quantization, attention kernels, and device mapping can be tuned specifically for AMD Radeon hardware.

3. Introduction to Core Capabilities

Capability	Description	Benefit
Hierarchical Planning	Breaks complex goals into ordered, concrete sub-tasks	Higher success rate on multi-step tasks
ReAct Reasoning	Thought → Action → Observation loop with robust tool calling	Transparent and controllable execution
Tool Use	Safe calculator, restricted Python interpreter, filesystem, local RAG	Real-world problem solving
Dual Memory	Short-term conversation buffer + persistent vector memory (Chroma)	Context continuity + long-term recall
Multi-Agent Routing	Automatic selection of Planner / Researcher / Coder / Executor	Specialization without manual configuration
Local RAG	Ingest and query private documents with semantic search	Privacy-preserving knowledge assistant
Fully Local Execution	No external LLM API required once the model is loaded	Data never leaves the machine

4. Model Introduction & Local Deployment Plan

Recommended Models (selected for consumer Radeon GPUs)

- **microsoft/Phi-3-mini-4k-instruct**: primary recommendation; excellent quality/size trade-off
- **Qwen/Qwen2.5-3B-Instruct**: strong multilingual and reasoning performance
- **meta-llama/Llama-3.2-3B-Instruct**: solid general-purpose alternative

Local Deployment Steps on AMD Radeon + ROCm

1. Install ROCm-compatible PyTorch:

```
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/rocm7.2
```

2. Install project dependencies:

```
pip install -e . && pip install -r requirements.txt
```

3. Download a model:

```
python scripts/download_models.py --model microsoft/Phi-3-mini-4k-instruct
```

4. Launch the system:

```
python -m src.ui.app # Gradio web interface
python examples/productivity_assistant.py # CLI demo
```

All runtime parameters (model name, quantization level, temperature, tool enablement, memory paths, etc.) are centralized in `config/default.yaml`. The LLM engine automatically detects the AMD GPU through the ROCm backend (exposed as `torch.cuda`) and falls back to CPU only when no compatible GPU is present.

5. Optimization for Inference Speed on AMD Radeon GPU

RadeonAgents contains multiple explicit optimizations aimed at maximizing tokens per second and minimizing VRAM consumption on AMD hardware:

Optimization	Implementation	Expected Impact
4-bit / 8-bit Quantization	bitsandbytes NF4 + double quantization	Fits larger models in 8–16 GB VRAM; higher throughput
SDPA Attention	attn_implementation="sdpa"	Better kernel efficiency on modern Radeon GPUs
Automatic Device Mapping	device_map="auto"	Optimal placement of model layers on the GPU
Controlled Generation	max_new_tokens, temperature, top_p	Avoids unnecessary computation
Lightweight Embeddings	all-MiniLM-L6-v2 for RAG & memory	Fast semantic search with low memory footprint
Benchmark Script	scripts/benchmark_rocm.py	Measures real tokens/s + VRAM on target GPU
Graceful Fallback	CPU path when no ROCm GPU detected	Still runnable for development and debugging

The dedicated benchmark script reports GPU name, VRAM allocated/reserved, tokens generated per second, and end-to-end latency. These concrete numbers can be shown in the demo video and evaluation materials to demonstrate ROCm optimization quality.

6. Summary of Deliverables

- Complete, well-structured source code with clear module separation
- Polished Gradio web interface + three ready-to-run example applications
- Local RAG pipeline over private documents
- Multi-agent orchestration with specialized roles (Planner, Researcher, Coder, Executor)
- ROCm-first inference engine with quantization, SDPA, and performance measurement
- Comprehensive README, configuration system, and unit tests

RadeonAgents demonstrates that sophisticated multi-agent systems featuring hierarchical planning, tool use, dual memory, and retrieval-augmented generation can run efficiently and privately on consumer AMD Radeon GPUs using the open ROCm software stack.

RadeonAgents — High-Level Architecture

